BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUCT DOCUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 PUBLIC CLASSIFICATION
Core Architecture & Development	**Technical Documentation**	Access Level: Public / Beta

---

Revision History

Version	Date	Author	Status	Description
1.0.0	2026-02-28	S. Köhne	Stable	Initial Release: ProWB Markdown Parser (GFM) & Security Spec

1. Introduction: ProWB Markdown Engine

The **ProWB Markdown Parser** (`md_parser.c`) is the proprietary text-rendering engine of the Pro Web Builder. Its primary function is the lossless, high-security conversion of documentation (such as manuals and audit reports) into W3C-compliant HTML5.

Since the ProWB is deployed in high-security environments (ProSuite Industrial), the parser was written from the ground up in strict C. It deliberately avoids vulnerable third-party regex libraries, relying instead on deterministic pointer arithmetic to guarantee maximum memory and execution safety. The engine fully supports the **GitHub Flavored Markdown (GFM)** industrial standard.

2. Security Architecture (Military Grade)

A parser handling external or dynamically generated files is a primary target for buffer attacks and code injections. This engine is hardened at the C-level against the following attack vectors:

2.1 Memory Bounds & Safety

- **Buffer Overrun Protection:** Strings and arrays are strictly constrained by hard boundaries. The maximum line length is hard-limited to `LINE_SIZE` (8,192 bytes), and tables are capped at 32 columns (`MAX_TABLE_COLS`). If an attacker attempts to inject oversized payloads, the C engine physically truncates the buffer to prevent memory corruption.
- **Buffer Underrun Protection:** Before any pointer arithmetic or array index access (e.g., `clean_name[2]`), the absolute minimum length of the string is validated via `strlen()`. Negative or insufficient lengths trigger an immediate and safe abortion of the function.
- **OOM (Out-of-Memory) Handling:** Every dynamic memory allocation (`malloc`) is explicitly checked for `NULL`. In the event of memory exhaustion, the rendering of the affected element is aborted safely, preventing a system-wide segmentation fault (Segfault).

2.2 XSS & Code Injection Prevention

- **ID-Sanitization (`_sanitize_for_id`):** Filenames and IDs intended for JavaScript events (`onclick`) are forced through an aggressive whitelist filter. Only alphanumeric characters (`A-Z`, `a-z`, `0-9`) and underscores (_) are permitted. Manipulated payload IDs such as `view_home')alert(1` are deterministically sanitized to `view_home___alert_1`, making it impossible to break out of the JavaScript string context.
- **HTML-Escaping (`_escape_html_string`):** Potentially dangerous control characters (`<`, `>`, `&`, `"`, `'`) within code blocks or generated labels are rigorously masked into their safe HTML entities (`<`, `>`, etc.) before being rendered.

3. Supported GFM Syntax (Reference)

The engine supports the following GitHub Flavored Markdown (GFM) specifications.

3.1 Headings

Headings from H1 to H6 are supported. **Important:** The `#` symbol must be followed by a mandatory space; otherwise, it is treated as standard text (e.g., a hashtag).

```
# Level 1 (H1)
## Level 2 (H2)
###### Level 6 (H6)
```

3.2 Text Formatting

Supports bold, italic, and strikethrough formatting, including combinations.

```
**Bold Text** or __Bold Text__
*Italic Text* or _Italic Text_
~~Strikethrough Text~~
```

3.3 Lists & Enumerations

The engine intelligently switches between ordered (`<ol>`) and unordered (`<ul>`) lists. Furthermore, task lists are natively converted into disabled HTML checkboxes.

```
- Unordered List (Dash)
* Unordered List (Asterisk)

1. Ordered List (Dot)
1) Ordered List (Parenthesis)
```

```
- [ ] Open Task
- [x] Completed Task
```

3.4 Blockquotes

```
> This is a highlighted blockquote.
> It can span across multiple lines safely.
```

3.5 Code Blocks & Inline Code

Inline code is marked by single backticks. Multi-line code blocks support language tags (e.g., `c`, `javascript`) to enable frontend syntax highlighting.

```
Use the `_malloc` function for safe memory allocation.

```c
int main() {
    printf("ProSuite Industrial");
    return 0;
}
```
```

(Note: Ensure the code block above is closed with actual backticks in your markdown file)

3.6 Links & Images

```
[Visit BrainAI](https://brainai.example)
![ProSuite Logo](./assets/logo.png)
```

3.7 Tables (with Alignment)

GFM tables are fully parsed. The colons within the separator line define the column alignment (Left, Center, Right).

```
| Module | Status | Metric |
| :--- | :---: | ---: |
| ProTU | Active | 48-52% |
| ProEDC | Active | 256-Bit |
```

3.8 Horizontal Rules

Generates a horizontal dividing line (`<hr>`).

```
---
```

© 2026 Sascha Köhne	✉️ **Contact for KYC: koehne83@googlemail.com**	🛡️ **ProTU-Verified**
System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**